



Introduction to GCSDK

SHAKTI Group | CSE Dept | PS-CDISHA - RISE Lab | IIT Madras

RISE LAB

IIT - Madras

Preview of the Session

1. GC2025 SoC's
2. Introduction to GCSDK
3. GCSDK Framework
4. Compiling and Executing 'Hello World' Program
5. Adding new Application to GCSDK
6. Adding new Peripheral to GCSDK
7. Adding a Project to GCSDK
8. Q & A

GC2025 SHAKTI SoC's



1. GANGA SoC

- Ganga is a 64-bit RISC-V SoC based on the SHAKTI C-Class core with a 6-stage pipeline, compliant with the RV64IMACSU instruction set.

2. YAMUNA SoC

- Yamuna is a 32-bit RISC-V SoC based on the SHAKTI C-Class core with a 6-stage pipeline, compliant with the RV32IMAC instruction set.

Introduction to GCSDK



Introduction to GCSDK

- Software Development Kit (SDK) provided to participants in GC2025.
- Clean separation between boot, drivers, core and application layers.
- Single place for bare-metal application development, projects and benchmarks.
- Driver support for PWM, SPI, PLIC, CLINT, UART, I2C, GPIO, GPTimer and XADC.
- Standalone and Debug mode supported.

Application to Output flow

- After the application is compiled , it is important to understand the flow of how the binary is executed. Tools like openocd and riscv-gdb helps in executing the compiled binary

1. Openocd (Open On-Chip Debugger) -

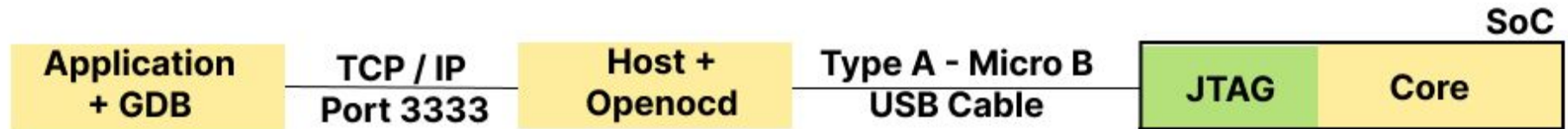
- Tool that lets the host interact with target hardware.
- Act as bridge between host and target hardware.



Application to Output flow

2. GDB (GNU Debugger) -

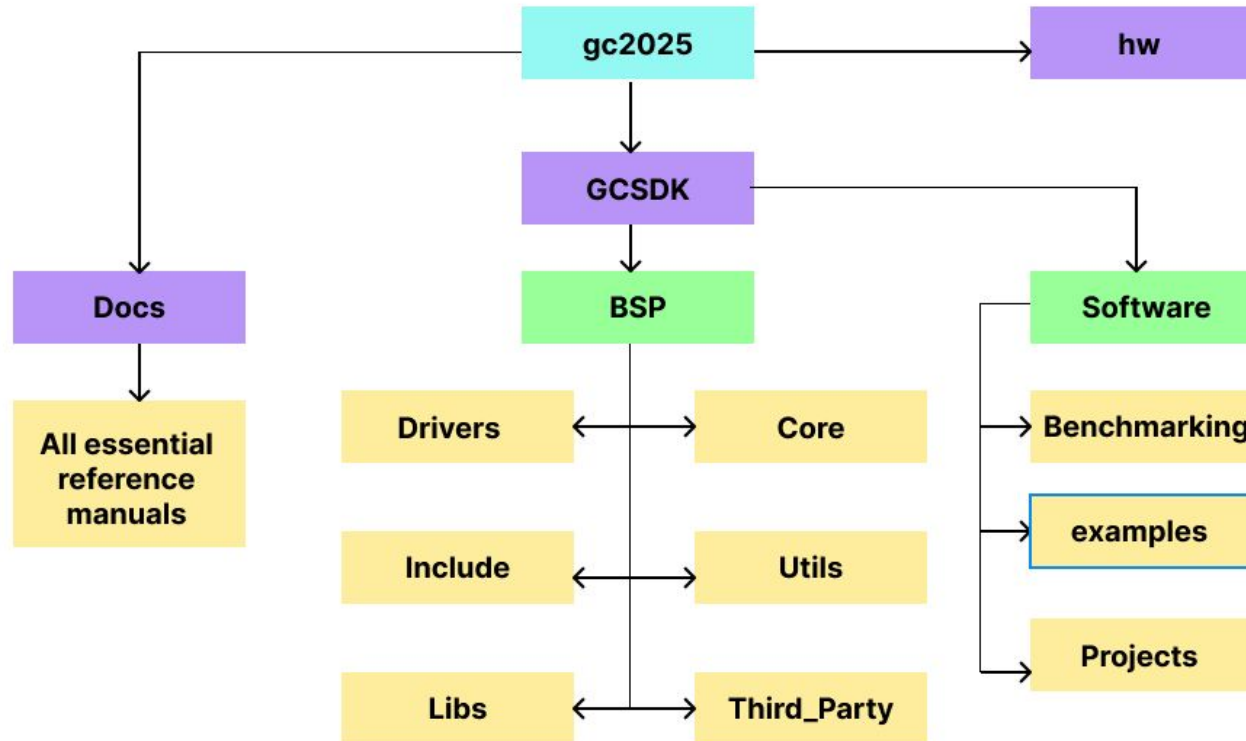
- Debugging tool designed for RISC V architecture.
- Program that helps to see what the code is doing inside the processor - while it runs, stops, crashes, or behaves weirdly.



Understanding Framework of GCSDK



GCSDK Framework



BSP - Board Support Package

The BSP is layer above the hardware. It consists of

- Driver files for various protocols (peripherals).
- Header files for driver files.
- Essential functions and reusable code like 'printf' and multi-level logging for debugging.
- Startup code , trap handler , memory initialisation.
- Files required for booting
- Platform dependent definitions tailored to each SoC.

Drivers

clint

ethernet

i2c

plic

pwm

spi

sspi

uart

xadc

- Drivers are for all supported peripherals (protocols).
- Inside each driver file is a set of software constructs (functions / API).
- Executes a particular task in hardware.
- Called by the application code / project.



Include

<code>#include clint_driver.h</code>	<code>#include pinmux.h</code>
<code>#include defines.h</code>	<code>#include plic_driver.h</code>
<code>#include eth_driver.h</code>	<code>#include pwm_driver.h</code>
<code>#include gpio.h</code>	<code>#include qspi.h</code>
<code>#include gpio_i2c.h</code>	<code>#include spi.h</code>
<code>#include gpio_spi.h</code>	<code>#include traps.h</code>
<code>#include i2c.h</code>	<code>#include uart.h</code>
<code>#include log.h</code>	<code>#include utils.h</code>
<code>#include memory.h</code>	<code>#include xadc_driver.h</code>

- Header files for all the peripheral drivers.
- Generic But Board specific.
- Peripheral registers are mapped as arrays.
- Instance of arrays used to access the registers.
- Macros for bit level description of the registers.

c log.c

c printf.c

c util.c

- Utilities are defined as APIs with definitions and implementations. e.g Printf, multi level log prints, delay loops, string operations, etc.
- Called repeatedly in the application codes. e.g. Print library, delay_loop.

<code>C</code> <code>init.c</code>	Trap Handling initialisations & calling the main function.
<code>[0]</code> <code>start.S</code>	Register initialisation, Section initialisation like (stack, text, global, data, etc.)
<code>[0]</code> <code>trap.S</code>	Pushing the register values into Stack, Call ISR, Pop the register values.
<code>C</code> <code>traps.c</code>	Find the cause for the trap and jump to the Handler

Core has implementation which

- Initialises memory.
- Trap handler functions to handle exceptions and interrupts.
- Startup file which initialises the RISC-V Registers.
- Handlers for different privilege levels.

Ganga
Yamuna

- Board specific address mapping (platform.h).
- Configuration files to be used for debugging (ftdi.cfg).
- Linker for various sections of the code.
- Support for backward compatibility.

 spanion

 burnFlash.py

 elf_to_header.c

- Board SPI or External SPI can be used for uploading the application code into flash.
- Application code to be flashed is converted into a header file before uploading.
- Has the required python scripts required to flash the application code.

examples	Example driver codes for peripherals.
projects	Application Project with many drivers.

- “Examples” consist of Application code targeted use the functionality of individual peripheral.
- “Projects” consists of Application code involving multiple peripherals.
- “BenchMarks” It is a set of Standard works , used to evaluate performance of the Core.

Examples

clint_applns

eth_test

gpio_applns

i2c_applns

malloc_test

plic_applns

pwm_applns

spi_applns

uart_applns

xadc_applns

- Simple code to show how the driver API can be called.
- Application code which uses only one peripheral (driver) as main one.
- Has multiple application code for each driver.



Projects

intruder_detection

irrigationsystem

lora

uart-cam

weatherstation

weatherstation_bmp280

- Integrates more than one peripheral driver.
- Application code which uses more than one peripheral.
- Focus here is to show how multiple drivers can be integrated into a single top file and used.

Demo : Compile and Execute “Hello World” Program

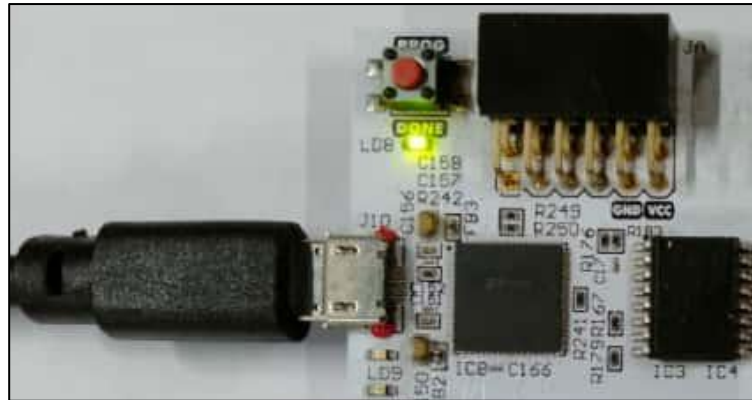
Prerequisite

- Vivado - to program the FPGA.
- OpenOCD - to configure the host as server.
- RISC-V-GNU-TOOLCHAIN - connect the core (SoC) as client to server and accesses the core/SoC.
- GTKTerm - for viewing the console prints.

Program the FPGA

- As demonstrated in previous session , program the FPGA with the bitstream / mcs using Vivado

After Programming the board



Refer the "Bitstream/MCS Generation and FPGA Target Configuration" manual in GCSDK/Docs

Compile the Application

- Make sure the working directory is /GCSDK
Execute the following command

```
make software PROGRAM=hello TARGET=<target_soc>
```

The <target_soc> can either be 'ganga' or 'yamuna'.
*please note <target_soc> is case sensitive

Refer the "Shakti SDK " user manual in GCSDK/Docs


```
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK$ make software PROGRAM=hello TARGET=yamuna
```

```
make[2]: Leaving directory '/home/rj/shakti/GRand_Challenge/gc2025/GCSDK/software/examples'
cd uart_applns/hello && make hello.riscv TARGET=yamuna DEBUG=
make[2]: Entering directory '/home/rj/shakti/GRand_Challenge/gc2025/GCSDK/software/examples/uart_applns/hello'
/home/rj/belsoc/bsoc_toolchain/build__rv32imac/bin/../../lib/gcc/riscv32-unknown-elf/13.2.0/../../../../riscv32-u
segment with RWX permissions
make[2]: Leaving directory '/home/rj/shakti/GRand_Challenge/gc2025/GCSDK/software/examples/uart_applns/hello'
All done !
make[1]: Leaving directory '/home/rj/shakti/GRand_Challenge/gc2025/GCSDK/software/examples'
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK$
```



Connecting Openocd

- Make sure the working directory is /GCSDK
Execute the following command

```
cd bsp/third_party/<Target_SoC>  
sudo ./openocd -f ftdi.cfg
```

The <target_soc> can either be 'ganga' or 'yamuna'.
*please note <target_soc> is case sensitive

Refer the "Shakti SDK " user manual in GCSDK/Docs

```
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK$ cd bsp/third_party/yamuna/
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK/bsp/third_party/yamuna$ ls
bootlink.ld ftdi.cfg link.ld openocd platform.h
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK/bsp/third_party/yamuna$ sudo ./openocd -f ftdi.cfg
[sudo] password for rj:
Open On-Chip Debugger 0.12.0+dev-03694-gca7d88252 (2024-02-29-16:07)
Licensed under GNU GPL v2
For bug reports, read
    http://openocd.org/doc/doxygen/bugs.html
DEPRECATED! use 'adapter speed' not 'adapter_khz'
DEPRECATED! use 'adapter driver' not 'interface'
DEPRECATED! use 'ftdi vid_pid' not 'ftdi_vid_pid'
DEPRECATED! use 'ftdi channel' not 'ftdi_channel'
DEPRECATED! use 'ftdi layout_init' not 'ftdi_layout_init'
Info : auto-selecting first available session transport "jtag". To override use 'transport select <transport>'.
Info : Simple Register based Bscan Tunnel Selected
Info : clock speed 1000 kHz
Info : JTAG tap: riscv.cpu tap/device found: 0x13631093 (mfg: 0x049 (Xilinx), part: 0x3631, ver: 0x1)
Info : [riscv.cpu.0] datacount=12 progbufsize=16
Info : [riscv.cpu.0] Disabling abstract command reads from CSRs.
Info : [riscv.cpu.0] Disabling abstract command writes to CSRs.
Info : [riscv.cpu.0] Examined RISC-V core
Info : [riscv.cpu.0] XLEN=32, misa=0x40001105
[riscv.cpu.0] Target successfully examined.
Info : [riscv.cpu.0] Examination succeed
Info : starting gdb server for riscv.cpu.0 on 3333
Info : Listening on port 3333 for gdb connections
riscv.cpu.0 halted due to debug-request.
Info : Listening on port 6666 for tcl connections
Info : Listening on port 4444 for telnet connections
Info : accepting 'gdb' connection on tcp/3333
riscv.cpu.0 halted due to debug-request.
Warn : Prefer GDB command "target extended-remote :3333" instead of "target remote :3333"
```



Connecting Debugger

- Make sure the working directory is /GCSDK
Execute the following command

```
riscv(XLEN)-unknown-elf-gdb -x gdb.script
```

*please note XLEN depends on the toolchain you have installed
RV64 : XLEN = 64 ; RV32 : XLEN = 32

Refer the "Shakti SDK " user manual in GCSDK/Docs

```
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK$ riscv32-unknown-elf-gdb
GNU gdb (GDB) 14.2
Copyright (C) 2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-pc-linux-gnu --target=riscv32-unknown-elf".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb) set remotetimeout unlimited
(gdb) target remote localhost:3333
Remote debugging using localhost:3333
warning: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
0x00001540 in ?? ()
(gdb) file software/examples/uart_applns/hello/output/hello.shakti
A program is being debugged already.
Are you sure you want to change the file? (y or n) y
Reading symbols from software/examples/uart_applns/hello/output/hello.shakti...
(gdb) load
Loading section .text.init, size 0xa0 lma 0x80000000
Loading section .text, size 0x4036 lma 0x800000a0
Loading section .text.strcpy, size 0x78 lma 0x800040d6
Loading section .rodata, size 0xab8 lma 0x80004150
Loading section .eh_frame, size 0x3c lma 0x80004c08
Loading section .sdata, size 0x8 lma 0x80004c44
Start address 0x80000000, load size 19530
Transfer rate: 48 KB/sec, 2790 bytes/write.
(gdb) c
```

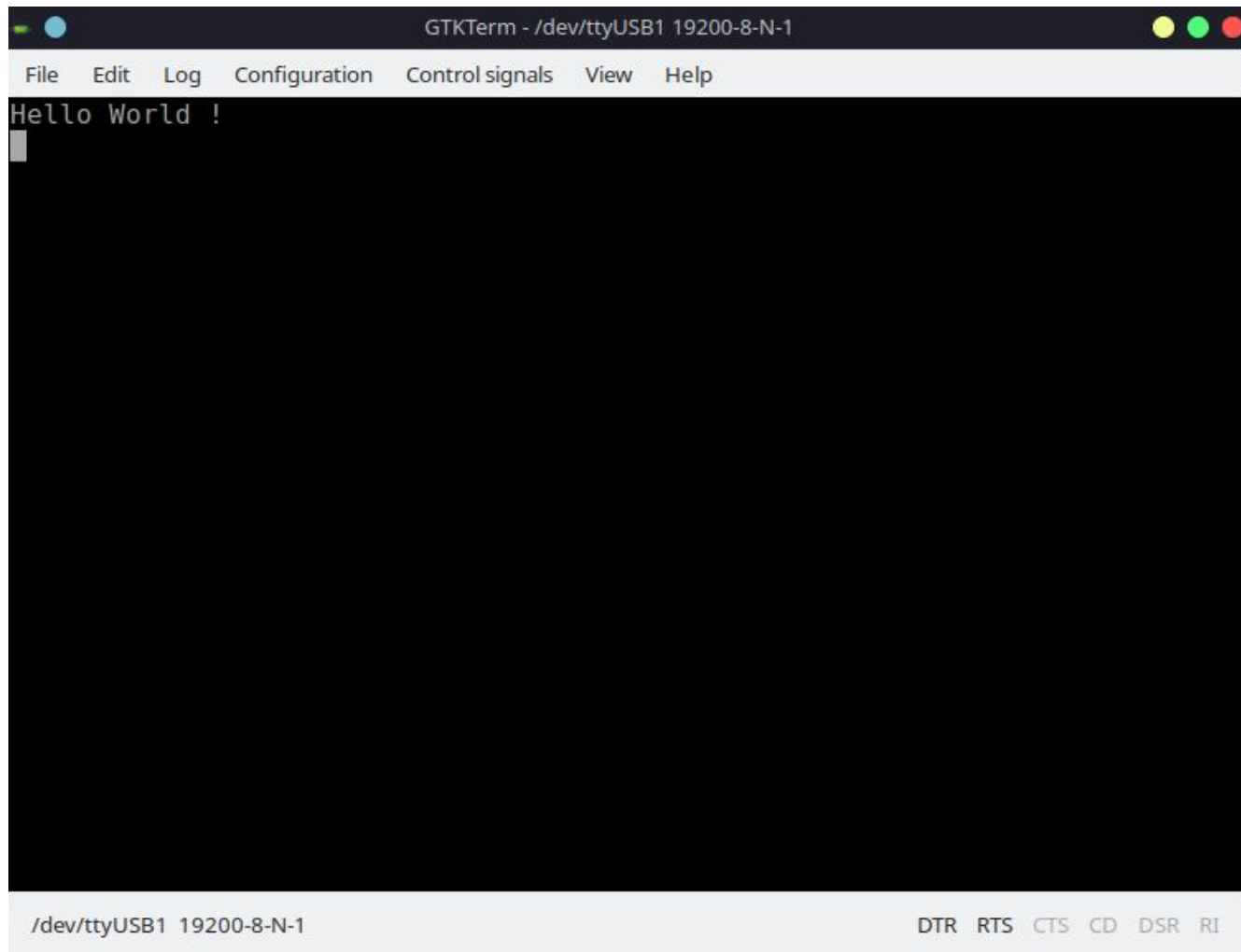


Viewing the Output

- Make sure the working directory is /GCSDK
Execute the following command

```
sudo gtkterm
```

Refer the “Shakti SDK ” user manual in GCSDK/Docs



References

Shakti Documentation: <https://shakti.org.in/documentation.html>

Shakti Blogs: <https://blogshakti.org.in/>

Shakti FPGA files: <https://gitlab.com/shaktiproject/sp2020>

Shakti SDK: <https://gitlab.com/shaktiproject/software/shakti-sdk>

Shakti on Arduino: <https://blogshakti.org.in/how-to-print-hello-world-on-shakti-using-arduino-ide/>

Linux on Shakti: <https://gitlab.com/shaktiproject/software/linux-on-shakti>

Platform IO: <https://registry.platformio.org/platforms/platformio/shakti>

Shakti cores: <https://gitlab.com/shaktiproject/cores>

Shakti peripherals: <https://gitlab.com/shaktiproject/uncore/devices>



Thank you

